



中国科学技术大学

University of Science and Technology of China

发先之极管

引脚输出电压给设备供电

2.2 GPIO输出电压

点灯

RoboWalker | Yssickjgd

目录



中国科学技术大学
University of Science and Technology of China

- 理论知识
 - 时钟
 - 通用输入输出端口
- 工程配置
- 编写代码
- 部分代码解析

1 理论知识

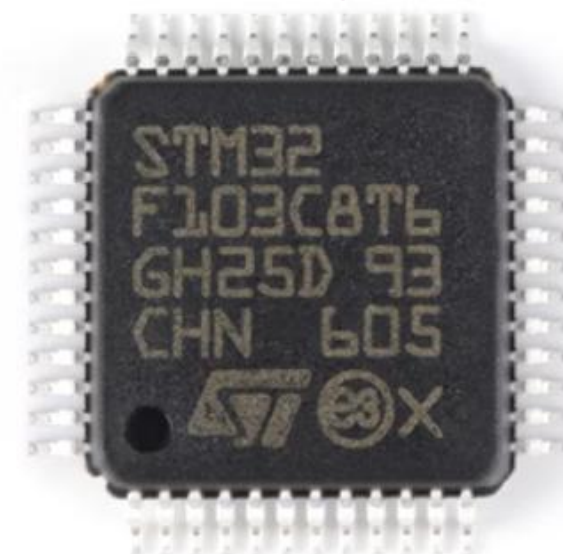


中国科学技术大学
University of Science and Technology of China

- 微型控制器单元

- Microcontroller Unit, MCU
- 类似于电脑的中央处理器 (Central Processing Unit, CPU)
- 是核心处理器，一般是开发板上的一个方形芯片，面积相对较大
- 一般而言是3.3V供电，能处理数字信号，低电平约为0V，高电平约为3.3V

0 1 信号 (一般规则: 低电平: 0, 高电平: 1)

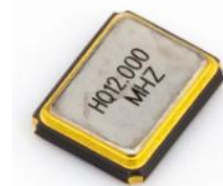


1.1 理论知识——时钟



中国科学技术大学
University of Science and Technology of China

- 要执行程序，就要有序，时序是很关键的，因此需要时钟，没时钟不行
- 时钟，是STM32单片机以及各种设备的心跳
- 时钟控制器（Reset Clock Control, RCC）
 - 有低速高速两种（High Speed, HS与Low Speed, LS）
 - 低速用于实时时钟（Real Time Clock, RTC）等
 - 高速用于定时器、UART、ADC等多种外设
 - 有内部外部两种（Internal, I与External, E）
 - Disable
 - 不用外部时钟，则自动（强制）使用MCU核心内部时钟
 - 一般是不准的代名词
 - Crystal/ceramic resonator（晶体/陶瓷振荡器）（如右图几种）
 - 使用开发板上的某个元件产生，该元件俗称晶振
 - 可以受一定激励产生时钟信号，经过MCU内部时钟电路驱动后可以作为单片机的时钟
 - Bypass clock source（旁路时钟源）
 - 由外部其他设备主动给出
 - 该时钟信号不经过MCU内部时钟电路驱动可直接作为单片机的时钟



1.1 理论知识——时钟



中国科学技术大学
University of Science and Technology of China

- 时钟树
 - 基本时钟信号会经过单片机MCU内部电路的处理，通过倍频、分频实现不同频率的子时钟
 - 不同的子时钟作用到不同的设备上，适配不同设备的工作环境
 - 按需配置，一般性能拉满

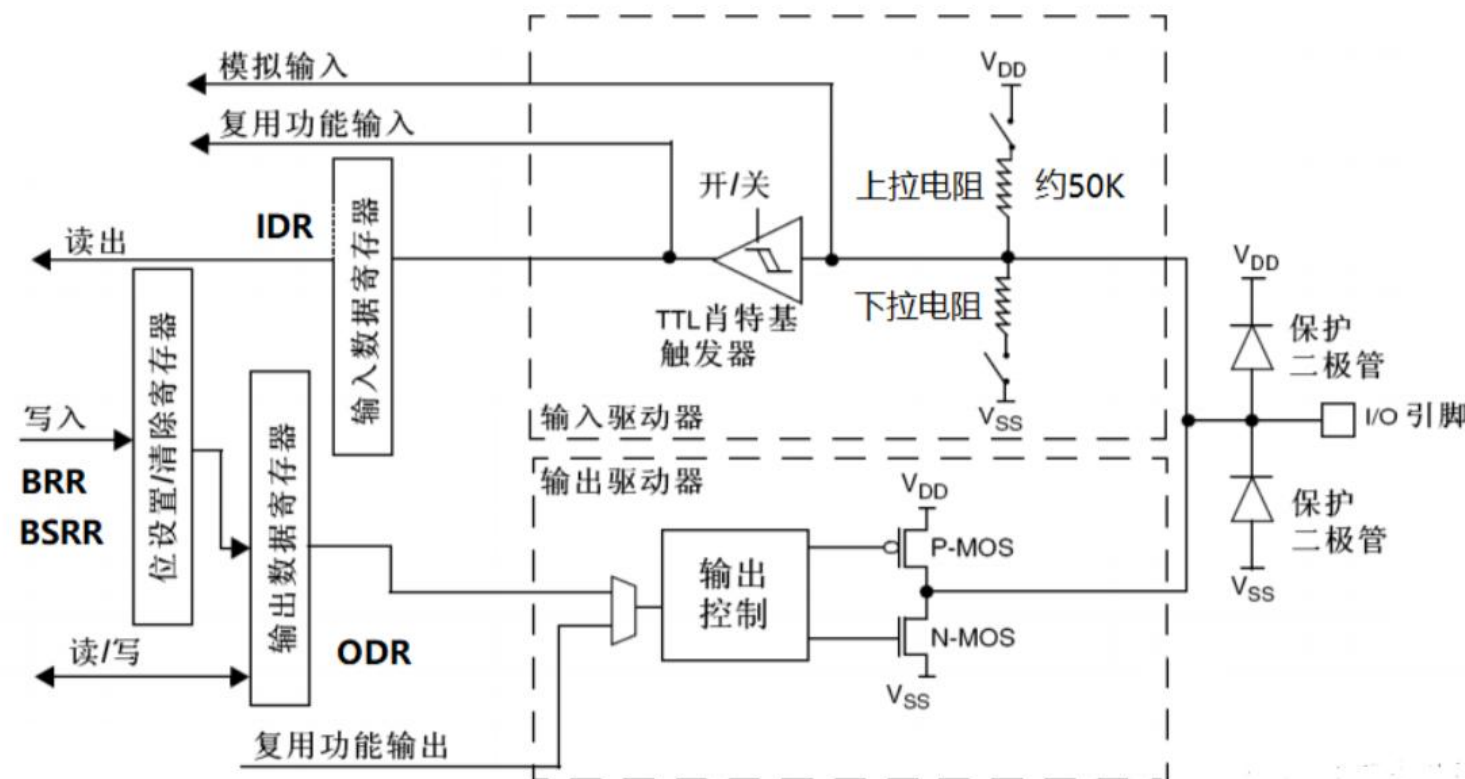
System Core -> RCC

1.2 理论知识——通用输入输出端口



中国科学技术大学
University of Science and Technology of China

- 有时钟就相当于有了一块开垦好的地，接下来就可以输入输出信号，也就是所谓的种庄稼摘果实
- 通用输入输出端口（General Purpose Input Output, GPIO），是MCU与外部元件沟通的接触点，一般沟通的是电压信息
- 俗称引脚。右图看不懂不要紧，以后回来看也行【插个眼】
- 引脚工作模式
 - 输入
 - 下拉输入
 - 上拉输入
 - 浮空输入
 - 模拟输入
 - 输出
 - 推挽输出
 - 开漏输出
 - 复用推挽输出
 - 复用开漏输出



1.2 理论知识——通用输入输出端口



中国科学技术大学
University of Science and Technology of China

- 下拉输入
 - 数字信号输入，只有高低电平两种
 - 等效在MCU内对引脚接一个电阻，这个电阻叫下拉电阻
 - 阻值具体多少不重要，反正比较大
 - 电阻另一端接MCU的低电平
 - 引脚没有任何输入时，此时读取引脚则是低电平
- 上拉输入
 - 数字信号输入，只有高低电平两种
 - 等效在MCU内对引脚接一个电阻，这个电阻叫上拉电阻
 - 阻值具体多少不重要，反正比较大
 - 电阻另一端接MCU的高电平
 - 引脚没有任何输入时，此时读取引脚则是高电平
- 浮空输入
 - 数字信号输入，只有高低电平两种
 - 不接入任何上拉下拉电阻
 - 引脚没有任何输入时，此时读取引脚电平未定，即悬而不定的状态，浮空
 - 万用表检测可能出现0到3.3V之间任意值
- 模拟输入
 - 前三个输入均是离散时间离散值的数字信号，也就是只有低电平和高电平
 - 模拟输入可以对连续时间连续值的模拟信号进行读取，当然也会在单片机中转化成可以被处理的离散时间离散值的数字信号
 - 具体内容将在本STM32CubeMX基础系列的ADC教程中讲述

1.2 理论知识——通用输入输出端口



中国科学技术大学
University of Science and Technology of China

- 推挽输出
 - 高低电平驱动能力都强
 - 宁折不弯，推挽高低电平相遇会烧坏
- 开漏输出
 - 低电平驱动能力强
 - 高电平驱动能力弱
 - 高电平通过自己主动外接上拉电阻实现输出
 - 输出强弱能力取决于上拉电阻与对应的上拉电平
 - 输出跳变速度取决于上拉电阻，电阻越小越快
 - 墙头草，开漏高低电平相遇会变低，&&
- 复用****输出
 - 用于外设等需要对引脚绑定某特定MCU内功能输出的情况

给设备供电的能力

2 工程配置

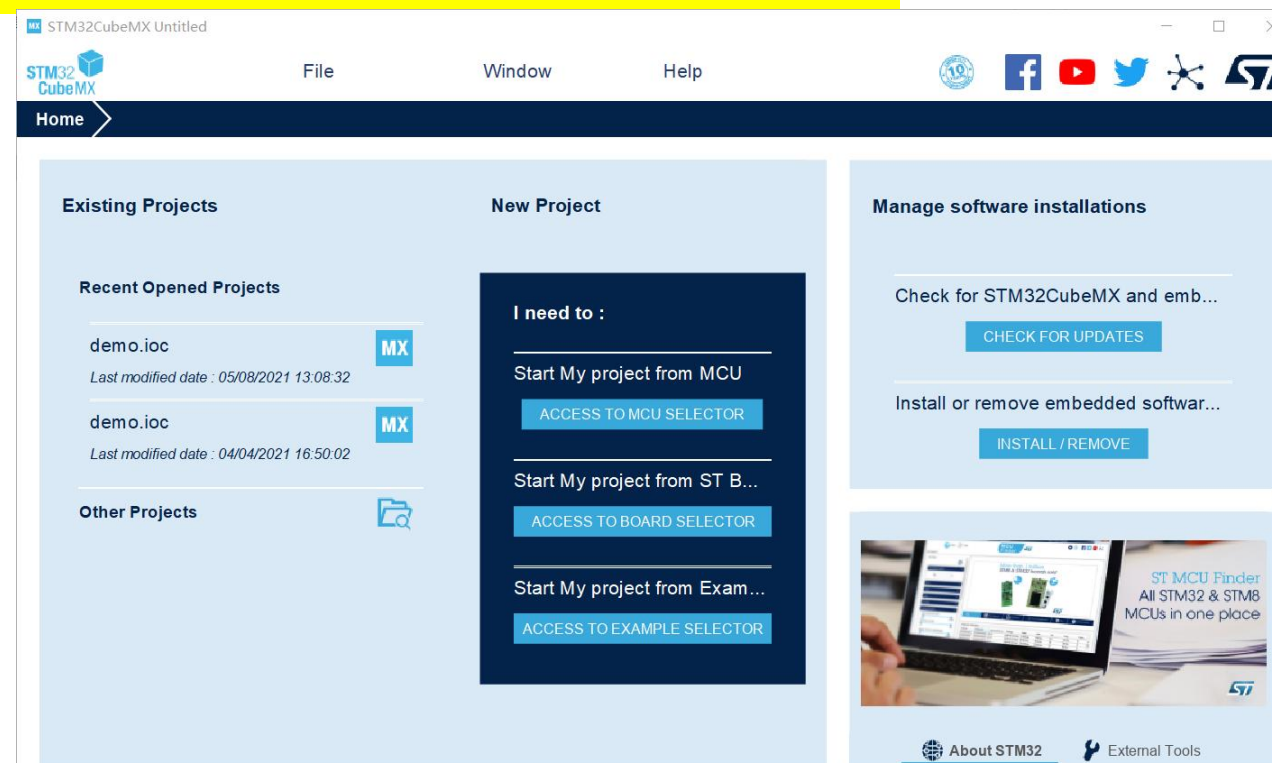


中国科学技术大学
University of Science and Technology of China

- 打开CubeMX
- 选择MCU芯片
- 选择自己手头的单片机MCU型号，双击

F103C8T6

F401CCU6

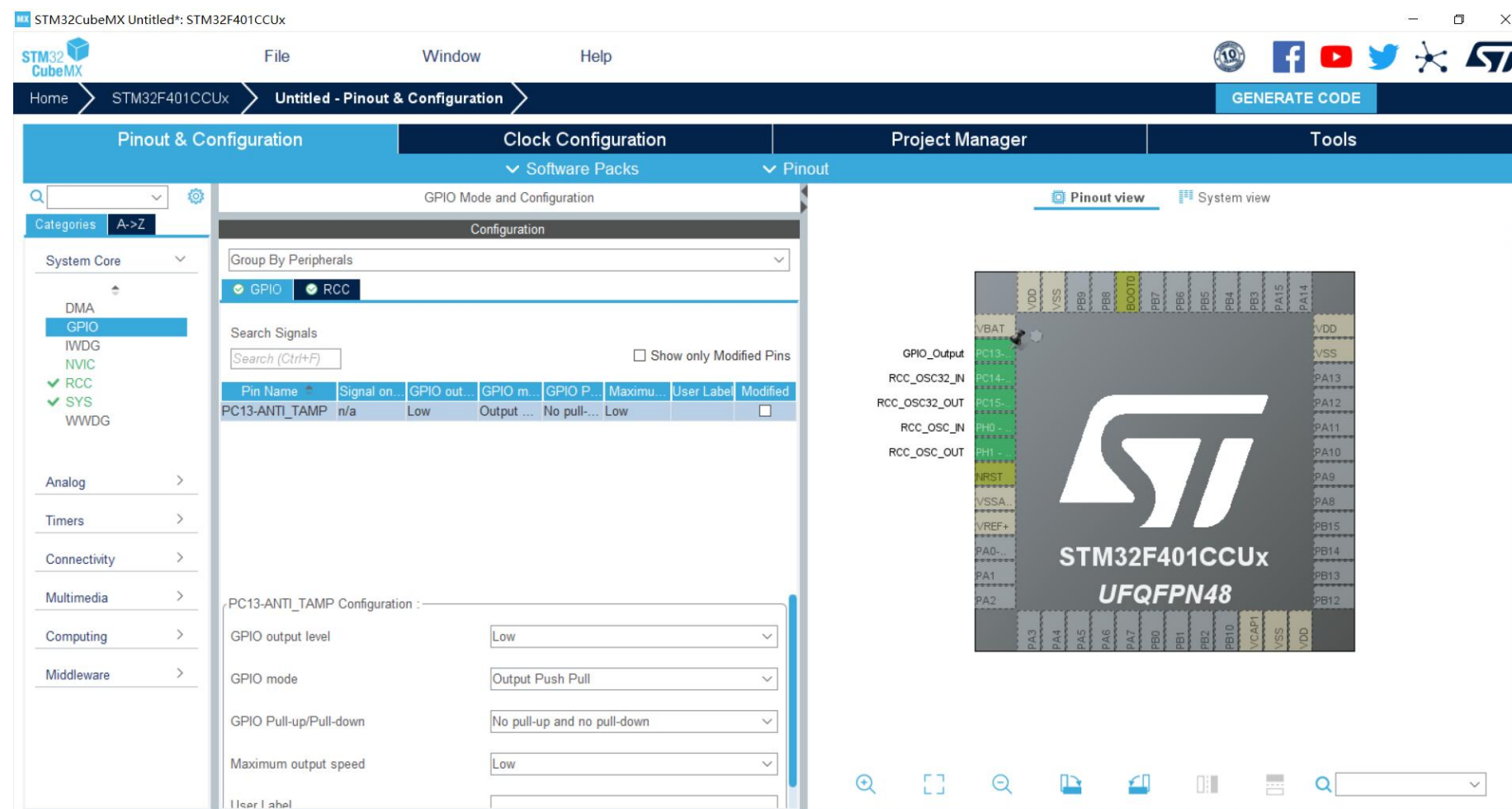


2 工程配置



中国科学技术大学
University of Science and Technology of China

- 第二个重要界面，工程配置界面
- 差不多长这样



2 工程配置



中国科学技术大学
University of Science and Technology of China

- 配置复位时钟控制器 (Reset Clock Control, RCC)
 - Crystal/ceramic resonator
- 配置时钟树
 - 根据开发板晶振频率设置HSE的频率，本开发板用的是25MHz的
 - 让主频通过HSE输出
 - 主频配置成50MHz (主要是为了好算)

2 工程配置



中国科学技术大学
University of Science and Technology of China

- 根据不同的烧写器，选择不同的调试模式
 - 本系列STM32CubeMX基础中，手头配套的是ST-link，因此选择serial wire
 - 如果用其他烧写器，请对应配置好

2 工程配置



中国科学技术大学
University of Science and Technology of China

- 根据原理图或PCB图，找到开发板上LED灯对应的单片机引脚。本系列教程STM32CubeMX基础实验采用的开发板中，LED灯对应的引脚为PC13
- 将PC13设置为输出模式
 - 初始电平为高电平（该引脚一开始是高电平，灯不亮）
 - 输出模式选择推挽输出



2 工程配置



中国科学技术大学
University of Science and Technology of China

- 工程管理设置中
 - 命名工程
 - 不要有中文等非ASCII码的字符
 - 最好以C语言变量要求
 - 修改IDE为MDK-ARM，对应我们的Keil这一套工具
- 代码生成中
 - 配置每个外设生成一个单独的头文件，便于管理
- 生成代码

2 工程配置



中国科学技术大学
University of Science and Technology of China

- 必要的开发前Keil配置

- 编译优化不是很智能

- 取消编译优化 *魔法棒 -> C/C++ -> Optimization 选择 Level 0 (-O0)*

- 让自己在学校学的“C语言”编译通过

- 我们学的其实是一种比较新的C语言，因此要打开C99模式，还有GNU Extension
- 有些语法其实是C++的但我们当C来学，所以要--cpp11

- 烧写即运行

- Reset and run *Debug -> Settings -> Flash Download*

- 代码初步认知

- 整体工程架构

- main.c的内容

- 其余必要文件的简要了解

*Startup - 初始化单片机系统的环境
Application/USER/core
Driver - 底层驱动内容
CMSIS - 指明了哪款单片机以及具体适配到单片机型号下的配置*

3 编写代码



中国科学技术大学
University of Science and Technology of China

- 主程序编写

```
91      /* Infinite loop */
92      /* USER CODE BEGIN WHILE */
93      while (1)
94      {
95          HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
96          HAL_Delay(500);
97          /* USER CODE END WHILE */
98
99          /* USER CODE BEGIN 3 */
100     }
101     /* USER CODE END 3 */
102 }
103
```

3 编写代码



中国科学技术大学
University of Science and Technology of China

- 选择合适的烧写器，取决于你手里的烧写器是啥
 - 本培训采用的是ST-link
- 编译
- 运行

自己写的代码要在 `begin` 和 `end` 中间

4 部分代码解析



- 我们的库代码本质上，都是对寄存器（内存）的读写
- 不同地址，表示不同设备的不同状态，可以读取也可以设置

```
425 -
426 - /**
427  * @brief Toggles the specified GPIO pins.
428  * @param GPIOx Where x can be (A..K) to select the GPIO peripheral for STM32F429X device or
429  *           x can be (A..I) to select the GPIO peripheral for STM32F40XX and STM32F427X devices.
430  * @param GPIO_Pin Specifies the pins to be toggled.
431  * @retval None
432  */
433 void HAL_GPIO_TogglePin(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)
434 {
435     uint32_t odr;
436
437     /* Check the parameters */
438     assert_param(IS_GPIO_PIN(GPIO_Pin));
439
440     /* get current Output Data Register value */
441     odr = GPIOx->ODR;
442
443     /* Set selected pins that were at low level, and reset ones that were high */
444     GPIOx->BSRR = ((odr & GPIO_Pin) << GPIO_NUMBER) | (~odr & GPIO_Pin);
445 }
```


4 部分代码解析



中国科学技术大学
University of Science and Technology of China

- 我们的库代码本质上，都是对寄存器（内存）的读写
- 很多参数，是宏定义，归根结底是一个地址或一个状态代表的数值

```
779 #define GPIOA      ((GPIO_TypeDef *) GPIOA_BASE)
780 #define GPIOB      ((GPIO_TypeDef *) GPIOB_BASE)
781 #define GPIOC      ((GPIO_TypeDef *) GPIOC_BASE)
782 #define GPIOD      ((GPIO_TypeDef *) GPIOD_BASE)
783 #define GPIOE      ((GPIO_TypeDef *) GPIOE_BASE)
784 #define GPIOH      ((GPIO_TypeDef *) GPIOH_BASE)
785 #define CRC         ((CRC_TypeDef *) CRC_BASE)
786 #define RCC         ((RCC_TypeDef *) RCC_BASE)
787 #define FLASH       ((FLASH_TypeDef *) FLASH_R_BASE)
788 #define DMA1        ((DMA_TypeDef *) DMA1_BASE)
```

```
690 /*!< AHB1 peripherals */
691 #define GPIOA_BASE      (AHB1PERIPH_BASE + 0x0000UL)
692 #define GPIOB_BASE      (AHB1PERIPH_BASE + 0x0400UL)
693 #define GPIOC_BASE      (AHB1PERIPH_BASE + 0x0800UL)
694 #define GPIOD_BASE      (AHB1PERIPH_BASE + 0x0C00UL)
695 #define GPIOE_BASE      (AHB1PERIPH_BASE + 0x1000UL)
696 #define GPIOH_BASE      (AHB1PERIPH_BASE + 0x1C00UL)
697 #define CRC_BASE        (AHB1PERIPH_BASE + 0x3000UL)
698 #define RCC_BASE        (AHB1PERIPH_BASE + 0x3800UL)
699 #define FLASH_R_BASE    (AHB1PERIPH_BASE + 0x3C00UL)
700 #define DMA1_BASE        (AHB1PERIPH_BASE + 0x6000UL)
701 #define DMA1_Stream0_BASE (DMA1_BASE + 0x010UL)
```

```
649 /*!< Peripheral memory map */
650 #define APB1PERIPH_BASE PERIPH_BASE
651 #define APB2PERIPH_BASE (PERIPH_BASE + 0x00010000UL)
652 #define AHB1PERIPH_BASE (PERIPH_BASE + 0x00020000UL)
653 #define AHB2PERIPH_BASE (PERIPH_BASE + 0x10000000UL)
654
```

```
632 /** @addtogroup Peripheral_memory_map
633  * @{
634  */
635 #define FLASH_BASE      0x08000000UL /*!< FLASH(up to 1 MB) base address in the alias region
636 #define SRAM1_BASE      0x20000000UL /*!< SRAM1(64 KB) base address in the alias region
637 #define PERIPH_BASE     0x40000000UL /*!< Peripheral base address in the alias region
638 #define SRAM1_BB_BASE   0x22000000UL /*!< SRAM1(64 KB) base address in the bit-band region
639 #define PERIPH_BB_BASE  0x42000000UL /*!< Peripheral base address in the bit-band region
640 #define BKPSRAM_BB_BASE 0x42480000UL /*!< Backup SRAM(4 KB) base address in the bit-band region
641 #define FLASH_END       0x0803FFFFUL /*!< FLASH end address
642 #define FLASH_OTP_BASE  0x1FFF7800UL /*!< Base address of : (up to 528 Bytes) embedded FLASH OTP Area
643 #define FLASH_OTP_END   0x1FFF7A0FUL /*!< End address of : (up to 528 Bytes) embedded FLASH OTP Area
644
```


感谢观看



中国科学技术大学
University of Science and Technology of China